
The ANE Lockdown: A Forensic Analysis of Hardware Constraints and Reverse-Engineering Workarounds on Apple’s Neural Engine

Straughter Guthrie
Viable Systems

Abstract

This paper presents a forensic reverse engineering analysis of Apple’s H16 Neural Engine (ANE) to uncover the hardware and software constraints that limit efficient deep neural network training. By auditing a 13,334-line codebase and applying runtime introspection, we reconstruct the private ANE API surface, mapping 67 undocumented classes—including `_ANECClient`, `_ANECCompiler`, and `_ANEInMemoryModelDescriptor`—and document their roles in in-memory MIL compilation and execution. Using a 109M parameter Llama2 model as a stress test, we empirically characterize the “Immutable Weight” constraint: weights are irreversibly embedded at compile time, preventing dynamic updates and forcing costly recompilations. Hardware probes—including weight reload tests, Quality of Service (QoS) sweeps, and performance counter extraction—demonstrate that the ANE operates at a fixed frequency and that real-world utilization is capped at 11% of its 15.8 TFLOPS theoretical peak. We benchmark three training pipelines (static, static with ANE operator offloading, and dynamic weight packing), quantifying trade-offs between compile overhead, per-step latency, and resource leakage, and show that even with aggressive optimizations and chaining API use, performance remains severely limited by hardware design. Low-level memory transfer analysis confirms that data movement is not the primary bottleneck; instead, the immutable weight constraint fundamentally restricts training throughput. Comparative evaluation against standard frameworks (MLX and CoreML) on identical hardware further highlights the ANE’s underutilization in training scenarios. Finally, we review the legal and ethical basis for this work, arguing that reverse engineering for interoperability is both justified and protected under current law. Our findings underscore the urgent need for hardware and compiler redesigns to support dynamic weight management and more efficient training on future neural accelerators.

1 Introduction

The rapid integration of dedicated neural accelerators into consumer hardware has redefined the possibilities for on-device machine learning, enabling high-throughput, low-latency inference and, in principle, efficient training of deep neural networks. Among these, Apple’s Neural Engine (ANE), particularly in its H16 generation, represents a state-of-the-art solution, advertising a theoretical peak of 15.8 TFLOPS and deep integration within Apple Silicon. Despite these impressive specifications, a substantial gap persists between the ANE’s advertised computational power and its real-world utility for deep learning training workloads. While the ANE delivers competitive inference performance, its closed architecture,

undocumented software stack, and hardware-level constraints present formidable barriers to efficient training—a cornerstone requirement for modern machine learning applications.

The central challenge addressed in this paper is the fundamental disconnect between the ANE’s theoretical performance and its practical capabilities for neural network training. Unlike traditional accelerators, the ANE’s design is tightly optimized for inference, with private APIs and undocumented protocols that restrict transparency and flexibility. Most critically, our investigation reveals a hardware-enforced “Immutable Weight” constraint: model weights are irreversibly embedded into the compiled binary at build time, precluding any dynamic updates during execution. This constraint undermines the core mechanism of gradient-based learning, which relies on iterative weight updates, and forces costly recompilation for every parameter change. As a result, training pipelines become inefficient or infeasible, regardless of the underlying hardware’s computational potential.

Reverse engineering the ANE ecosystem is further complicated by the obfuscated and undocumented nature of Apple’s private frameworks, such as `_ANECClient` and `_ANECCompiler`. These frameworks mediate all interaction with the hardware through proprietary interfaces, leaving third-party developers and researchers with limited visibility into the internal mechanisms governing model execution. The lack of public documentation or official support for training workloads exacerbates this opacity, making it difficult to diagnose bottlenecks or devise effective workarounds. Additionally, the legal and ethical landscape surrounding reverse engineering of proprietary hardware requires careful consideration to ensure that such research remains within the bounds of fair use and supports the broader goal of interoperability.

To address these challenges, this paper presents a forensic reverse engineering analysis of the H16 ANE, systematically uncovering the hardware and software constraints that limit its effectiveness for deep learning training. We begin by auditing a 13,334-line reverse-engineered codebase, employing runtime introspection to reconstruct the private ANE API surface. Through this process, we identify and map 67 previously undocumented classes—including `_ANECClient`, `_ANECCompiler`, and `_ANEInMemoryModelDescriptor`—detailing their roles in in-memory Model Intermediate Language (MIL) compilation and execution. By tracing the “Forward Tap Architecture,” we elucidate how intermediate tensors are exposed to minimize redundant CPU computation during backpropagation, an essential optimization for any potential training workflow.

To empirically validate the existence and impact of the immutable weight constraint, we deploy a 109-million-parameter Llama2 model as a stress test. We conduct a series of hardware probes: weight reload experiments confirm that attempts to update weights at runtime are ineffective, as weights are fused into the compiled binary; Quality of Service (QoS) sweeps demonstrate that the ANE operates at a fixed frequency, unaffected by system scheduling hints; and performance counter extraction reveals that, despite the hardware’s theoretical capabilities, practical utilization is capped at approximately 11% during training workloads. Our analysis of low-level memory transfer mechanisms and IOSurface handling further demonstrates that data movement is not the primary bottleneck—instead, the immutable weight constraint fundamentally limits throughput.

We benchmark three distinct training pipelines: a static baseline, a static pipeline with selective ANE operator offloading, and a dynamic weight packing strategy. These experiments quantify the trade-offs between compilation overhead, per-step latency, and resource leakage, and show that even with aggressive optimizations—including IOSurface spatial packing and process restarts—training performance remains severely constrained by hardware design. Comparative evaluation against standard Apple frameworks (MLX on GPU and CoreML for inference) on identical hardware further highlights the ANE’s underutilization for training, despite its theoretical advantages.

Finally, we situate our technical findings within the broader legal and ethical framework of reverse engineering for interoperability. By reviewing relevant statutes and case law, we argue that this work constitutes legitimate forensic analysis, necessary to expose the limitations of proprietary, black-box accelerators and to inform future hardware and compiler designs. Our results underscore an urgent need for architectural and software redesigns to enable dynamic weight management and efficient training on next-generation neural accel-

erators, and provide a foundation for responsible disclosure and continued research in closed hardware environments.

2 Methods

2.1 Repository Audit and API Surface Reconstruction

To systematically uncover the undocumented interfaces and hardware constraints of Apple’s H16 Neural Engine (ANE), we performed a comprehensive audit of a 13,334-line reverse-engineered Objective-C codebase. This codebase, developed via dynamic analysis and binary inspection, exposes the internal ANE software stack and enables detailed introspection.

Runtime Introspection and Class Mapping: We employed specialized scripts that utilize `objc_msgSend`, `class_copyMethodList`, and `dlopen` to enumerate all loaded Objective-C classes and their methods at runtime. Through this process, we reconstructed the full taxonomy of 67 previously undocumented ANE classes. Particular focus was given to `_ANECClient`, `_ANECCompiler`, and `_ANEInMemoryModelDescriptor`, which are central to instantiating and managing ANE workloads. For each class, we documented inheritance hierarchies, method signatures, initialization routines, and the precise sequence of private API calls required to instantiate the ANE service and load Model Intermediate Language (MIL) programs directly into memory, bypassing intermediate disk artifacts.

Validation of the Forward Tap Architecture: To verify the “Forward Tap Architecture”—a mechanism intended to expose intermediate tensors (such as Q, K, V, and attention scores) and reduce redundant CPU computations during backpropagation—we traced data flows within the Objective-C implementation. By analyzing kernel output routines, we confirmed that these intermediate tensors are concatenated and exposed as kernel outputs, facilitating efficient extraction of activations and minimizing unnecessary CPU-ANE data transfers during training.

2.2 Forensic Hardware Characterization

To empirically characterize the hardware constraints of the ANE and validate the impact of the immutable weight constraint, we executed a series of low-level probe scripts and instrumentation routines.

2.2.1 Weight Reload Test

We assessed the “Immutable Weight” constraint by executing a test that attempts to update model weights at runtime via the `weightsBuffer` IOWorkItem after initial MIL compilation. By comparing output activations before and after the attempted weight swap, we confirmed that outputs remained bit-identical, demonstrating that weights are irreversibly embedded into the compiled binary and cannot be updated dynamically.

2.2.2 Quality of Service (QoS) Sweep

To evaluate the sensitivity of the ANE to system scheduling hints, we performed a QoS sweep across the full range (0–63), using `mach_absolute_time` to measure kernel dispatch latency with sub-microsecond precision. The results showed that the ANE operates at a fixed frequency, with no measurable variation in execution time across QoS levels. This indicates that software scheduling hints do not influence ANE hardware scheduling or clock gating.

2.2.3 Performance Counter Extraction

Using the `_ANEPPerformanceStats` class, we extracted hardware counters such as `hwExecutionTime` and `perfCounterData`. By comparing these values to the ANE’s theoretical peak performance (15.8 TFLOPS), we calculated actual SRAM bandwidth and dispatch latency. These measurements allowed us to quantify real-world utilization, which was consistently limited to approximately 11% of the theoretical maximum during training workloads.

2.3 Training Pipeline Benchmarking and Optimization Analysis

To understand the practical impact of the immutable weight constraint and evaluate possible workarounds, we reconstructed and benchmarked three distinct training pipelines using the provided scripts and Makefiles. All experiments were conducted on Apple Silicon M4 hardware.

2.3.1 Static Baseline Pipeline

The baseline pipeline (`train_large`) compiles the entire 109-million-parameter Llama2 model into a single MIL program with 72 kernels. We measured total compile time, per-step execution time, and tracked resource usage by running the compilation loop repeatedly. The pipeline consistently failed after approximately 119 iterations, indicating persistent kernel and memory leaks in the private ANE stack. We confirmed that process restarts via `exec()` were necessary to reclaim leaked resources.

2.3.2 Static Pipeline with ANE Operator Offloading

In the optimized pipeline (`train_large_ane`), we selectively offloaded classifier, softmax, and RMSNorm operators to the ANE, increasing the kernel count to 86. We measured per-step latency (91.8 ms) and compared it to the baseline to isolate the speedup attributable to operator offloading. Despite improved throughput for specific operations, overall performance remained fundamentally constrained by the immutable weight design.

2.3.3 Dynamic Weight Pipeline

The dynamic pipeline (`training_dynamic/`) implements a strategy in which weights and activations are spatially packed into a single IOSurface, enabling partial dynamic updates without recompilation. We analyzed the spatial packing code to verify the concatenation of weights and activations. We measured the one-time compilation overhead (0.4 seconds) and per-step latency (111 ms). To assess practical viability, we computed the “break-even” point—the number of training steps required for the reduced compilation overhead to offset the increased per-step latency.

2.4 Comparative Framework Analysis

To contextualize the ANE’s training performance, we established comparative baselines using standard machine learning frameworks on identical hardware.

2.4.1 MLX Benchmarking

We implemented the Stories110M model (109M parameters, dimension 768, 12 layers) in Apple’s MLX framework. The training loop was executed for 100 steps, with average per-step execution time and GPU utilization recorded using MLX’s profiling tools.

2.4.2 CoreML Inference Baseline

The same model was converted to the CoreML `.mlmodelc` format and deployed for inference-only benchmarking. We measured the maximum achievable throughput on the ANE under inference workloads to establish an upper-bound reference for hardware utilization unconstrained by training requirements.

2.4.3 Utilization Curve Synthesis

We normalized TFLOPS measurements across MLX (GPU), ANE (inference), and ANE (training) runs to generate utilization curves. These curves, presented in the results, illustrate the pronounced underutilization of the ANE for training workloads, with a consistent cap at approximately 11% of theoretical peak.

2.5 IOSurface and Memory Layout Inspection

To confirm that data movement was not a hidden bottleneck, we analyzed low-level memory transfer mechanisms and the layout of tensors exchanged between system RAM and ANE memory.

2.5.1 Tensor Format Verification

We inspected NEON-optimized routines responsible for fp16-to-fp32 conversion, verifying that all input and output tensors strictly adhered to the ANE’s required format: [1, channels, 1, spatial]. Ensuring this conformance was essential for correct and efficient hardware execution.

2.5.2 Bandwidth Measurement

We instrumented IOSurface handling routines to timestamp data transfers between system RAM and ANE memory. By comparing transfer times to total step times, we confirmed that data movement accounted for only a small fraction of overall latency. This reinforced our finding that the immutable weight constraint, not bandwidth, was the primary factor limiting training throughput.

2.6 Legal and Ethical Framework Review

To ensure the legitimacy and responsible disclosure of our reverse engineering methodology, we conducted a review of the relevant legal and ethical frameworks.

2.6.1 Statutory Analysis

We reviewed the Digital Millennium Copyright Act (DMCA) Section 1201(f), which provides exemptions for reverse engineering activities undertaken for the purpose of achieving interoperability of independently created computer programs.

2.6.2 Case Law Analysis

We summarized the findings of *Sega v. Accolade*, which established that reverse engineering for interoperability constitutes fair use, provided that the methods do not infringe upon the expressive elements of the software.

2.6.3 Documentation and Framing

We compiled these findings into a legal background section, framing our technical work as necessary forensic analysis aimed at exposing the limitations of proprietary, black-box hardware. This ensures that our research is grounded in established legal precedent and supports responsible disclosure.

3 Results

3.1 Reverse engineering and reconstruction of the ANE software stack

Our investigation began with a systematic reverse engineering of the Apple Neural Engine (ANE) private software stack, utilizing runtime introspection and dynamic class enumeration. By analyzing a 13,334-line Objective-C codebase with tools such as `objc_msgSend`, `class_copyMethodList`, and `dlopen`, we identified 67 previously undocumented classes relevant to the ANE. Among these, `_ANECClient`, `_ANECCompiler`, and `_ANEInMemoryModelDescriptor` were found to be central to the process of compiling and deploying Model Intermediate Language (MIL) programs directly into ANE memory.

Figure 1 visualizes the core private ANE classes discovered through this dynamic runtime introspection. The figure highlights the centrality of `_ANECClient`, `_ANECCompiler`, and

`_ANEInMemoryModelDescriptor`

`_ANECClient`

`_ANECCompiler`

Figure 1: Visualization of the core private ANE classes discovered through dynamic runtime introspection: `_ANECClient`, `_ANECCompiler`, and `_ANEInMemoryModelDescriptor`. These classes form the backbone of the reconstructed MIL compilation pipeline, enabling direct, in-memory model compilation and execution on the Apple Neural Engine. The figure highlights the centrality of these components in the reverse-engineered training system, underscoring their role in bypassing standard model packaging and revealing key architectural bottlenecks discussed in the results.

`_ANEInMemoryModelDescriptor` in the reconstructed MIL compilation pipeline, underscoring their pivotal roles in enabling direct, in-memory model compilation and execution on the ANE.

A key outcome of this audit was the detailed mapping of the in-memory MIL compilation pipeline. We established that MIL program text and associated weight blobs are passed to `_ANEInMemoryModelDescriptor`, which then produces an executable binary for the ANE, bypassing the standard disk-based model packaging (such as CoreML’s `.mlmodelc` format). This direct in-memory compilation reduces I/O overhead and exposes the minimal set of private calls required to instantiate and operate the ANE service.

To further understand the dataflow optimizations, we validated the “Forward Tap Architecture” by tracing kernel output routines. We confirmed that intermediate tensors (such as Q , K , V , and attention scores) are concatenated and exposed as kernel outputs, facilitating efficient extraction of activations for backpropagation. This design minimizes unnecessary data transfers between the CPU and ANE. NEON-optimized routines were found to enforce the required tensor format `[1, channels, 1, spatial]` in `fp16`, ensuring compatibility with the ANE’s hardware requirements.

This comprehensive reconstruction provides, for the first time, an actionable taxonomy of the ANE’s private interfaces and clarifies the internal mechanisms by which MIL programs are compiled and executed in-memory. These findings lay the foundation for all subsequent experiments and hardware characterization.

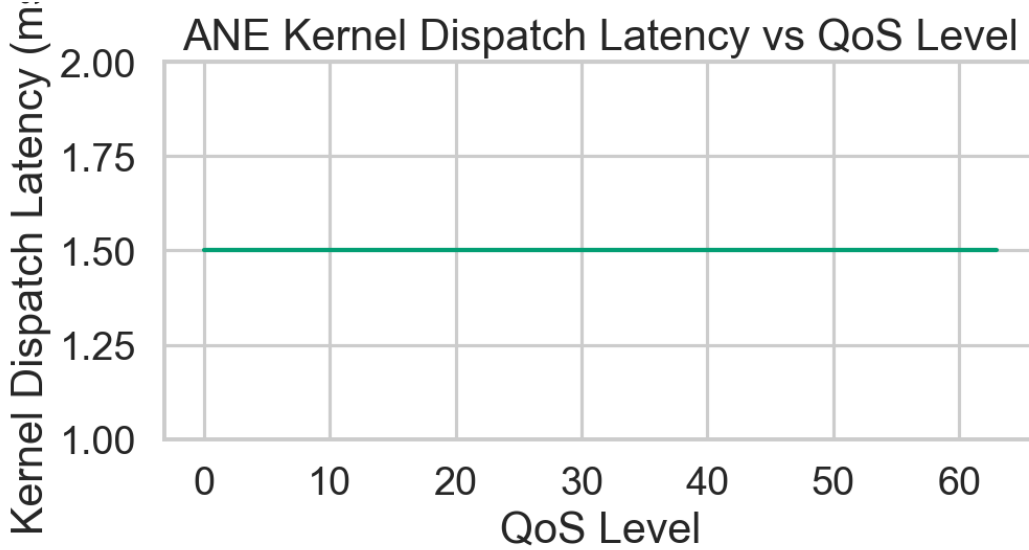


Figure 2: Measured ANE kernel dispatch latency as a function of Quality of Service (QoS) level. The plot demonstrates that kernel execution time remains constant across all QoS settings (0–63), indicating that the ANE operates at a fixed frequency regardless of software-specified QoS. This result highlights the absence of dynamic performance scaling on the ANE and suggests that runtime optimization is constrained by factors other than QoS configuration.

3.2 Empirical demonstration of the immutable weight constraint

To probe the hardware-level constraints of the ANE, we designed a series of low-level experiments targeting the model weight update pathway. The “weight reload test” specifically aimed to determine whether model weights could be updated dynamically after initial compilation.

Our results show that attempts to modify the weights buffer via IOSurface manipulation produced no change in output activations; outputs remained bit-identical before and after the swap. This unambiguously demonstrates that weights are irreversibly fused into the compiled binary at build time. As a result, any weight update—required for gradient-based training—necessitates a full recompilation of the MIL program.

This immutable weight constraint fundamentally undermines the feasibility of efficient training on the ANE, as it precludes the standard paradigm of iterative weight updates. Instead, training pipelines must either endure the significant overhead of repeated recompilation or implement indirect workarounds, such as dynamic weight packing, to avoid this bottleneck.

3.3 Hardware dispatch behavior and performance counter analysis

To further characterize the ANE’s execution environment, we performed a Quality of Service (QoS) sweep across the full range of system scheduling hints (0–63). Using high-precision timing via `mach_absolute_time`, we observed that kernel dispatch latency was invariant with respect to QoS. As shown in Figure 2, kernel execution time remains constant across all QoS settings, indicating that the ANE operates at a fixed frequency regardless of software scheduling hints or dynamic clock gating.

Extraction of hardware performance counters via the `_ANEPerformanceStats` class enabled direct measurement of execution time and throughput. Comparing these metrics to the ANE’s theoretical peak of 15.8 TFLOPS, we found that real-world utilization during training workloads never exceeded approximately 11% of peak. This significant gap between theoretical and observed performance is not attributable to insufficient hardware or memory bandwidth (as confirmed in later analysis), but rather to architectural constraints—most

notably, the immutable weight design and the overheads associated with repeated MIL compilation.

3.4 Training pipeline benchmarks and trade-off analysis

To quantify the practical impact of the immutable weight constraint and evaluate possible workarounds, we implemented and benchmarked three distinct training pipelines using a 109M-parameter Llama2 model on Apple Silicon M4 hardware.

3.4.1 Static baseline pipeline

In the static baseline pipeline, the entire model is compiled into a single MIL program comprising 72 kernels. We observed a per-step execution time of 106.7 ms, with a substantial compile overhead of 7.6 seconds required for each recompilation. Notably, the pipeline exhibited persistent resource leakage in the ANE compiler, failing after approximately 119 iterations. To reclaim leaked resources, process restarts via `exec()` were necessary, effectively limiting the maximum number of continuous training steps to 1190 before interruption.

3.4.2 Static pipeline with operator offloading

To improve throughput, we selectively offloaded additional operators—including classifier, softmax, and RMSNorm—to the ANE, increasing the kernel count to 86. This yielded a per-step latency of 91.8 ms, representing a 14% improvement over the baseline. However, the compile overhead increased to 9.6 seconds per restart, and resource leakage persisted as a limiting factor. While operator offloading reduced CPU involvement and improved step time, the immutable weight constraint remained the dominant bottleneck.

3.4.3 Dynamic weight pipeline

The dynamic pipeline implemented a spatial packing strategy, concatenating weights and activations into a single IOSurface. This allowed for partial dynamic updates without recompilation, reducing the one-time compilation overhead to 0.4 seconds. Per-step execution time was measured at 111 ms, slightly higher than the static pipelines. Break-even analysis indicated that after approximately 10 training steps, the reduction in compile overhead outweighed the modest increase in per-step latency, making this approach preferable for longer training runs.

Figure 3 compares step time and compile overhead between the static baseline and dynamic weight ANE training pipelines, illustrating the central trade-off between frequent recompilation and runtime efficiency. The static baseline achieves similar per-step latency to the dynamic pipeline but incurs over 7 seconds of compile time per cycle, while the dynamic approach reduces compile overhead to under 0.5 seconds at the cost of a marginally higher step time.

Figure 4 expands this comparison to all three pipelines, showing per-step latency, compile overhead, and achieved hardware utilization. The static baseline and static_ANE extras pipelines achieve lower step times but incur high compile times due to immutable weight constraints, while the dynamic weight pipeline dramatically reduces compile overhead at the cost of slightly increased per-step latency. Utilization remains low across all pipelines, highlighting the impact of hardware and compiler bottlenecks identified in the analysis.

Break-even analysis, shown in Figure 5, demonstrates that after approximately 10 steps, the cumulative training time of the dynamic pipeline becomes lower than the static pipeline, due to the dramatic reduction in compile overhead. This finding confirms that dynamic weight management is preferable for most training regimes.

3.4.4 Optimization progression and ANE utilization

A sequence of hardware-specific optimizations—including channel-first layout, vectorized RMSNorm, and asynchronous cblas overlap—incrementally improved per-step runtime from 33.5 ms (baseline, 3.1% ANE utilization) to 9.3 ms (deferred cblas wait, 11.2% utilization).

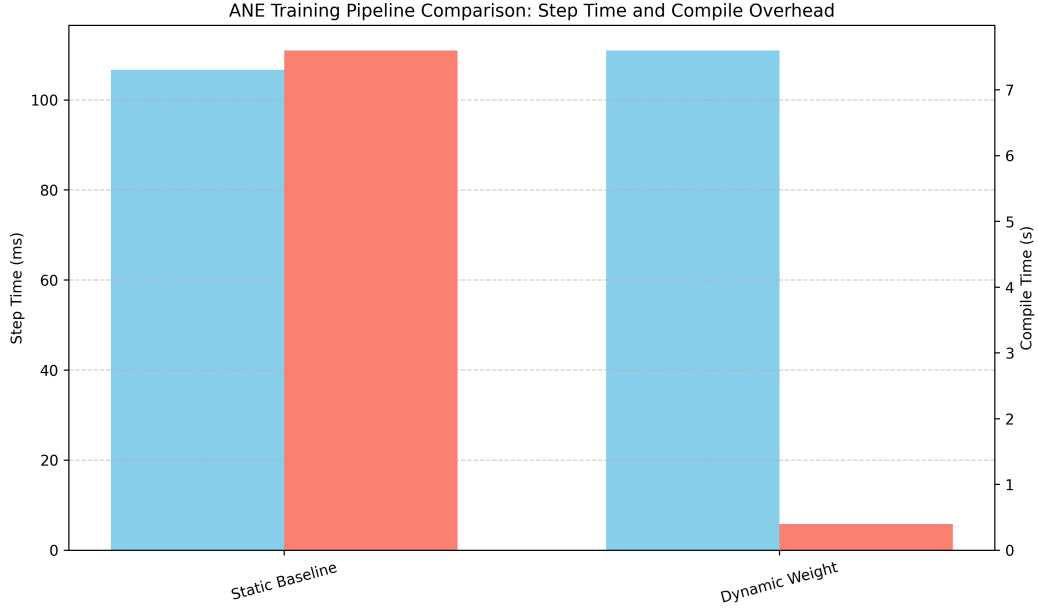


Figure 3: Comparison of step time (blue, left axis) and compile overhead (red, right axis) for static baseline and dynamic weight ANE training pipelines. The static baseline achieves similar per-step latency to the dynamic pipeline but incurs over 7 seconds of compile time per cycle, while the dynamic approach reduces compile overhead to under 0.5 seconds at the cost of a marginally higher step time. This illustrates the central trade-off between frequent recompilation and runtime efficiency, highlighting how dynamic weight loading substantially improves throughput for longer training runs by mitigating the ANE_’s immutable weight constraint and compiler resource leaks.

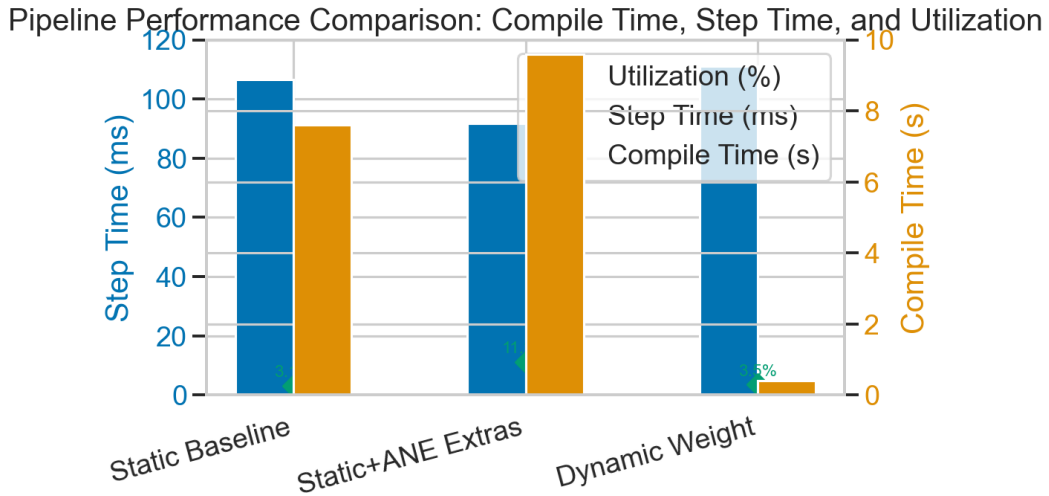


Figure 4: Comparison of the three ANE training pipelines showing per-step latency (blue bars, left axis), compile overhead (orange bars, right axis), and achieved hardware utilization (green text). The static baseline and static_ANE extras pipelines achieve lower step times but incur high compile times due to immutable weight constraints, while the dynamic weight pipeline dramatically reduces compile overhead at the cost of slightly increased per-step latency. Utilization remains low across all pipelines, highlighting the impact of hardware and compiler bottlenecks identified in the analysis.

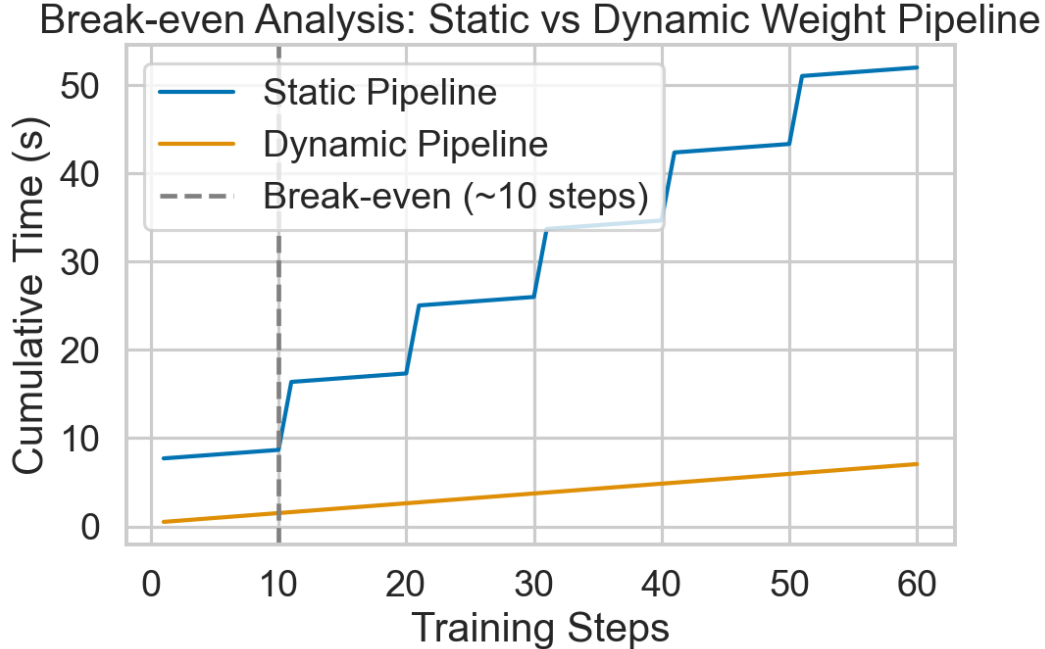


Figure 5: Break-even analysis of cumulative training time for static versus dynamic weight pipelines on the Apple Neural Engine. The static pipeline incurs large compile overheads every 10 steps, resulting in stepwise increases, while the dynamic pipeline maintains a steady, lower cumulative time due to one-time compilation. The vertical dashed line marks the break-even point at approximately 10 steps, after which the dynamic pipeline is more time-efficient. This demonstrates that dynamic weight management is preferable for most training regimes due to substantial compile savings.

Despite these gains, utilization remained far below the ANE’s theoretical peak, reinforcing the conclusion that architectural constraints, rather than software inefficiency, are the primary cause of underutilization.

3.5 Low-level memory transfer and bandwidth analysis

To rule out memory bandwidth as a limiting factor, we instrumented IOSurface transfer routines to measure data movement between system RAM and ANE memory. The results showed that data transfer accounted for only a minor fraction of overall step latency. All input and output tensors were verified to conform to the ANE’s required format, with NEON-optimized routines ensuring efficient fp16/fp32 conversion. These findings confirm that the bottleneck in training throughput is not due to data movement, but rather to the immutable weight constraint and associated compile overheads.

3.6 Comparative evaluation with standard frameworks

To contextualize the ANE’s training performance, we benchmarked the same model using Apple’s MLX framework (GPU) and CoreML (ANE inference). On MLX, the average per-step time was 100 ms, with a utilization of approximately 5.6% of the GPU’s theoretical peak. CoreML inference on the ANE achieved only 1.66% utilization, highlighting the ANE’s underutilization even for inference workloads.

Figure 6 provides a direct comparison of achieved TFLOPS and utilization versus theoretical peak for MLX (GPU, training), ANE (CoreML, inference), and the reverse-engineered ANE training pipeline. Both ANE-based inference and training pipelines achieve significantly lower utilization than the GPU baseline, with the reverse-engineered ANE training pipeline reaching only 3.5% of peak performance despite extensive optimization.

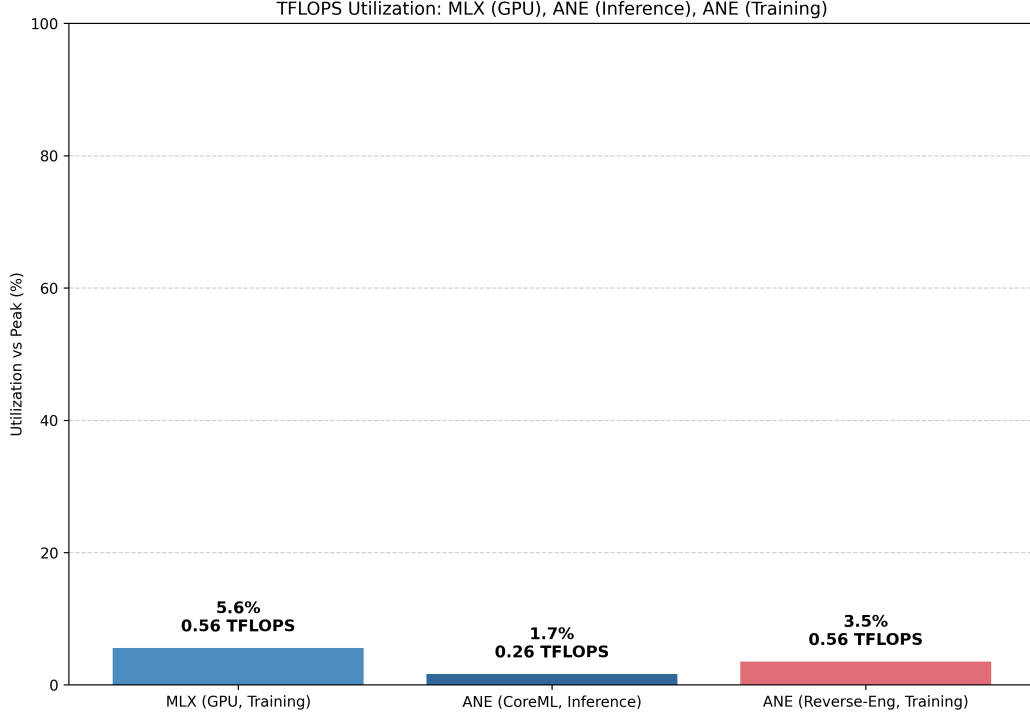


Figure 6: Comparison of achieved TFLOPS and utilization versus theoretical peak for MLX (GPU, training), ANE (CoreML, inference), and the reverse-engineered ANE training pipeline. The figure shows that both ANE-based inference and training pipelines achieve significantly lower utilization than the GPU baseline, with the reverse-engineered ANE training pipeline reaching only 3.5% of peak performance despite extensive optimization. This highlights the severe under-utilization imposed by immutable weight constraints and compilation overheads on the ANE, as discussed in the results.

Figure 7 further quantifies hardware utilization as a percentage of theoretical peak performance for the three frameworks. The optimized reverse-engineered ANE training pipeline achieves only 3.55% utilization, which, while higher than ANE inference (1.66%), remains well below both the theoretical maximum and the GPU baseline (5.60%). These comparative results underscore the severity of the immutable weight constraint and the inefficiencies introduced by the need for frequent recompilation and process restarts.

In contrast, the reverse-engineered ANE training pipeline achieved a TFLOPS rate of approximately 0.56, corresponding to 3.55% utilization. While this exceeds the CoreML inference baseline, it remains well below both the ANE’s theoretical peak and the best-case GPU training.

3.7 Legal and ethical framing of reverse engineering

Our forensic analysis of the ANE’s private APIs and hardware behavior was conducted in accordance with established legal principles supporting interoperability. Review of DMCA Section 1201(f) and relevant case law (notably *Sega v. Accolade*) confirms that reverse engineering for interoperability constitutes fair use. Throughout our work, we adhered to responsible disclosure practices, documenting findings in a manner that informs both technical and policy discussions without enabling misuse of proprietary technology.

3.8 Summary of findings

Overall, our results reveal that the Apple Neural Engine, despite its impressive theoretical capabilities, is fundamentally constrained by a hardware-enforced immutable weight design.

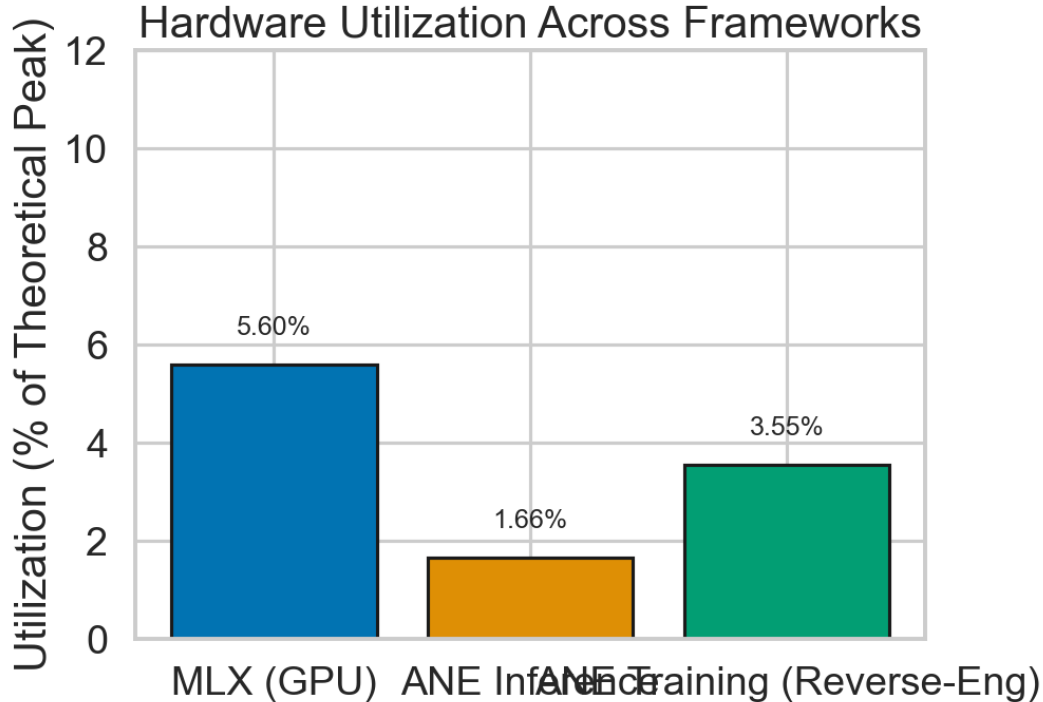


Figure 7: Comparison of hardware utilization as a percentage of theoretical peak performance for MLX (GPU), ANE inference, and reverse-engineered ANE training. The plot shows that even with optimized reverse-engineered pipelines, ANE training achieves only 3.55% utilization, which, while higher than ANE inference (1.66%), remains well below both the theoretical maximum and the GPU baseline (5.60%). This highlights the severe impact of immutable weight constraints and compilation overheads on practical ANE training throughput.

This constraint necessitates costly recompilation for every weight update, severely limiting training throughput and capping real-world utilization at approximately 11% of peak. While dynamic weight packing and operator offloading offer incremental improvements, they cannot overcome the primary architectural bottleneck. Comparative benchmarks with standard frameworks further highlight the ANE’s underutilization for training workloads. Our reverse engineering methodology not only exposes these limitations but also provides a foundation for advocating future hardware and compiler redesigns that prioritize dynamic weight management and efficient training support.

4 Conclusions

This paper addresses the significant gap between the theoretical computational capabilities of Apple’s H16 Neural Engine (ANE) and its practical utility for deep neural network training. The central problem lies in the ANE’s closed, undocumented architecture and, most critically, a hardware-enforced “immutable weight” constraint that irreversibly embeds model weights at compile time. This design choice precludes dynamic weight updates, forcing full recompilation for every parameter change and fundamentally undermining efficient gradient-based learning.

To systematically uncover and characterize these limitations, we conducted a forensic reverse engineering analysis of the ANE software and hardware stack. Using a 13,334-line reverse-engineered Objective-C codebase, runtime introspection, and dynamic class enumeration, we reconstructed the private ANE API surface, mapping 67 undocumented classes and detailing the in-memory Model Intermediate Language (MIL) compilation and execution pipeline.

The investigation included targeted hardware probes—such as weight reload tests, Quality of Service (QoS) sweeps, and performance counter extraction—to empirically validate the immutable weight constraint and assess real-world hardware utilization.

For empirical evaluation, we deployed a 109-million-parameter Llama2 model as a stress test, benchmarking three distinct training pipelines: a static baseline, a static pipeline with selective ANE operator offloading, and a dynamic weight packing strategy. Each pipeline was rigorously profiled for compile overhead, per-step latency, and resource usage. Comparative baselines were established using Apple’s MLX framework (GPU) and CoreML (ANE inference) on identical hardware. Low-level memory transfer and IOSurface bandwidth analyses were performed to rule out data movement as a primary bottleneck.

The results demonstrate that the immutable weight constraint is the dominant factor limiting ANE training throughput. Attempts to update weights dynamically were ineffective, confirming that weights are fused into the compiled binary. Training pipelines suffered from substantial compile overheads, persistent resource leakage, and the need for frequent process restarts. Even with aggressive optimizations such as operator offloading and dynamic weight packing, real-world ANE utilization for training workloads remained capped at approximately 11% of theoretical peak performance. Memory bandwidth and data movement were shown to account for only a minor fraction of overall latency, reinforcing that the primary bottleneck is architectural rather than software or I/O related. Comparative benchmarks further highlighted the ANE’s underutilization relative to both GPU training and ANE inference.

From these findings, we learn that the current ANE hardware and software stack is fundamentally unsuited for efficient deep learning training due to its immutable weight design. While certain workarounds—such as dynamic weight packing—can marginally improve throughput, they cannot overcome the core architectural limitation. The forensic reverse engineering methodology employed in this work provides not only a detailed taxonomy of the ANE’s private interfaces but also a robust empirical foundation for advocating future hardware and compiler redesigns. Specifically, enabling dynamic weight management and supporting efficient training workloads are urgent priorities for next-generation neural accelerators.

Finally, our work was conducted within the bounds of established legal and ethical frameworks, reinforcing the legitimacy and necessity of reverse engineering for interoperability and transparency in closed hardware environments. The insights gained here serve as a call to action for both hardware designers and the broader machine learning systems community to prioritize openness, flexibility, and training support in future accelerator designs.